

ESTRUTURA DE DADOS

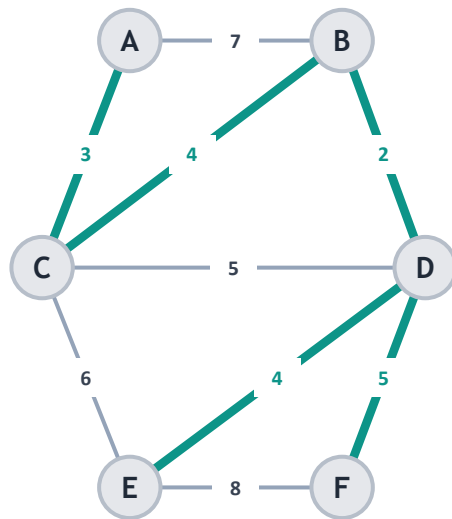
Árvore Geradora Mínima

Conceito, algoritmos de Kruskal e Prim, complexidade e aplicações

Prof. Dr. Antonio Marcos Selmini

antonio.selmini@espm.br

<https://github.com/espm-ed-2026>



AGM em destaque (peso 18)

CONCEITO

O que é a AGM?

Dado um grafo conexo, não direcionado e ponderado $G = (V, A)$, uma **árvore geradora** é um subgrafo que conecta todos os vértices sem formar ciclos. A **AGM (MST)** é a árvore geradora de menor peso total possível.

Geradora

Inclui todos os $|V|$ vértices do grafo.

Acíclica

Não contém ciclos — é uma árvore.

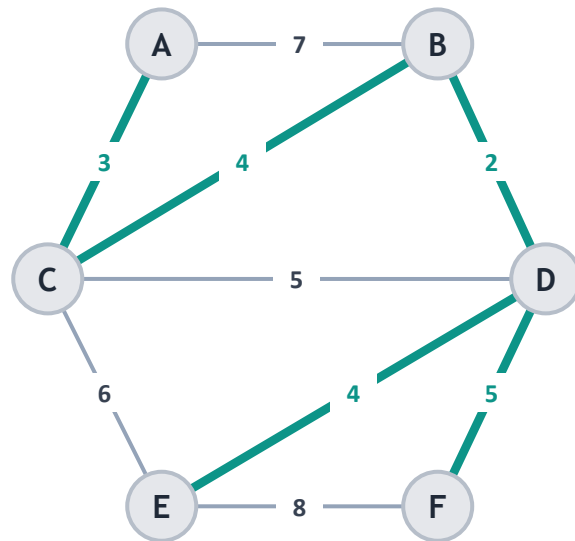
$|V| - 1$ arestas

Toda árvore geradora tem exatamente $n-1$ arestas.

Peso mínimo

A soma dos pesos das arestas é a menor possível.

Grafo G e sua AGM (arestas em teal)



Peso total da AGM = 2+3+4+4+5 = 18

Para que serve a AGM?

A AGM responde à pergunta: como conectar todos os pontos gastando o mínimo? É a base de diversos problemas reais de projeto e otimização de redes.

Redes de telecomunicação

Cabear bairros/torres conectando todos com o menor comprimento de fibra ou custo de enlace.

Redes elétricas e água

Projetar linhas de transmissão ou tubulações ligando todas as estações com custo mínimo.

Transporte e logística

Planejar malhas viárias/ferroviárias que liguem todas as cidades minimizando obras.

Agrupamento (clustering)

Remover as arestas mais caras da AGM separa os dados em grupos naturais.

Análise de imagens

Segmentação e reconstrução 3D usam AGM para conectar regiões similares.

Aproximação do TSP

A AGM fornece um limite inferior e heurísticas para o caixeiro-viajante.

ALGORITMO 1

Algoritmo de Kruskal

Estratégia gulosa orientada a arestas, com Union-Find para evitar ciclos.

Definição e regras

Ideia central: ordene todas as arestas por peso crescente e vá adicionando a mais barata que **não forme ciclo**, até obter $|V| - 1$ arestas.

- 1 Ordenar** — Liste todas as arestas em ordem crescente de peso.
- 2 Selecionar a menor** — Pegue a próxima aresta de menor peso ainda não examinada.
- 3 Testar ciclo** — Se ela conecta dois vértices de componentes diferentes, adicione-a à AGM.
- 4 Descartar se fecha ciclo** — Se os dois vértices já estão na mesma componente, descarte (formaria ciclo).
- 5 Repetir** — Continue até a AGM ter $|V| - 1$ arestas.

Estrutura-chave: Union-Find

find(x): descobre a componente de x.

union(x,y): funde duas componentes.

Mesma raiz $\Rightarrow$ adicionar a aresta fecharia ciclo.

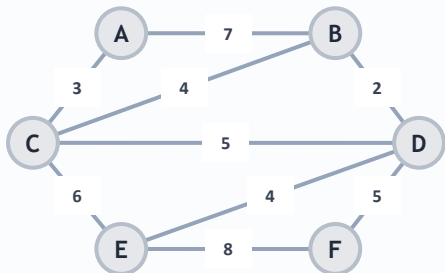
É guloso e correto

A cada passo escolhe o ótimo local (aresta mais barata segura); o resultado global é a AGM.

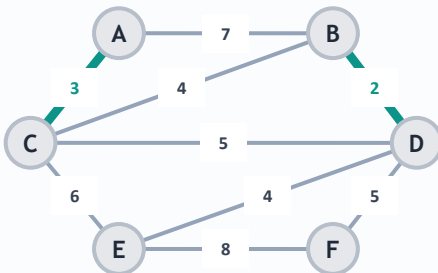
Execução em um grafo

Arestas ordenadas: B-D(2) · A-C(3) · B-C(4) · D-E(4) · C-D(5) · D-F(5) · C-E(6) · A-B(7) · E-F(8)

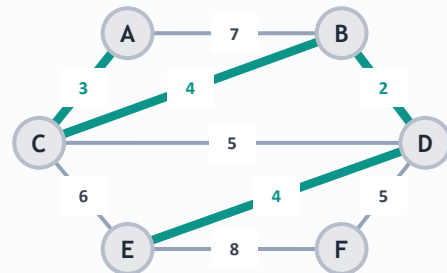
Passo 0 — nenhuma aresta



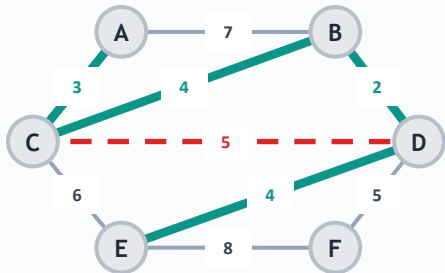
① B-D (2) ② A-C (3)



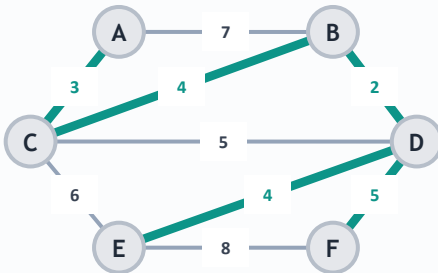
③ B-C (4) ④ D-E (4)



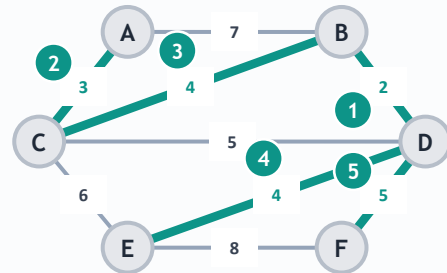
C-D (5): rejeita (ciclo B-C-D)



⑤ D-F (5) — AGM completa



Resultado · peso = 18



Pseudocódigo e complexidade

Pseudocódigo

```

KRUSKAL( $G = (V, A)$ ):
   $AGM \leftarrow \emptyset$ 
  para cada  $v$  em  $V$ :
    MAKE-SET( $v$ )           // cada nó isolado
  ordenar  $A$  por peso crescente
  para cada  $(u, v)$  em  $A$  (nessa ordem):
    se  $FIND(u) \neq FIND(v)$ : // não fecha ciclo
       $AGM \leftarrow AGM \cup \{(u, v)\}$ 
      UNION( $u, v$ )
    se  $|AGM| = |V| - 1$ :
      parar
  retornar  $AGM$ 

```

Complexidade

Ordenar arestas	$O(A \log A)$
-----------------	---------------

$ V $ operações MAKE-SET	$O(V)$
--------------------------	--------

2A FIND + V UNION	$O(A \cdot \alpha(V))$
-------------------	------------------------

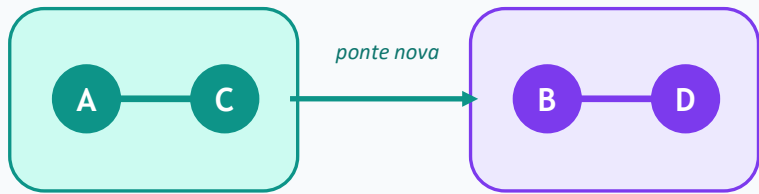
Total dominante	$O(A \log A)$
------------------------	---------------------------------

Como $\log A \approx \log V$ (pois $A \leq V^2$), também se escreve **$O(A \log V)$** . α é a função inversa de Ackermann — praticamente constante.

Componentes: a ideia das ilhas

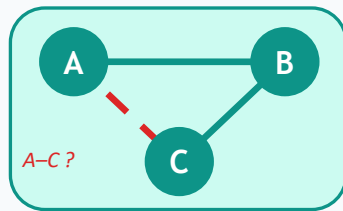
Cada grupo de vértices já ligados entre si é uma **componente** — pense nela como uma **ilha**. No início, cada vértice é uma ilha sozinha; cada aresta aceita é uma ponte que funde duas ilhas.

Ilhas diferentes → constrói ponte



$\text{FIND}(C) \neq \text{FIND}(B) \rightarrow$ representantes diferentes. A ponte não fecha ciclo: **adiciona e funde as ilhas.**

Mesma ilha → fecharia ciclo



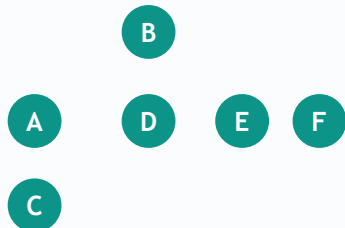
$\text{FIND}(A) = \text{FIND}(C) \rightarrow$ mesmo representante. Já há caminho $A \rightarrow B \rightarrow C$; a aresta fecharia ciclo: **descarta.**

O teste, em uma linha: compare os representantes das pontas. $\text{FIND}(u) \neq \text{FIND}(v) \rightarrow$ ilhas distintas, adicione e $\text{UNION}(u, v)$. Iguais $\rightarrow$ mesma ilha $\rightarrow$ descarte.

As ilhas se fundem aresta a aresta

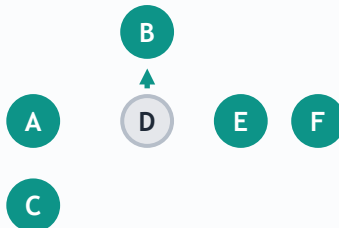
Cada conjunto é uma árvore; a raiz (em teal) é o representante. A seta aponta do filho para o pai. MAKE-SET cria as ilhas; UNION liga uma raiz à outra; FIND sobe até a raiz.

MAKE-SET — 6 ilhas



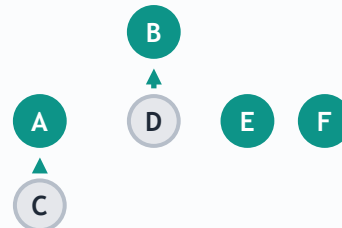
cada nó é seu representante

UNION(B,D) · B-D (2)



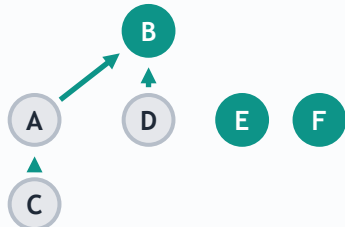
D passa a apontar para B

UNION(A,C) · A-C (3)



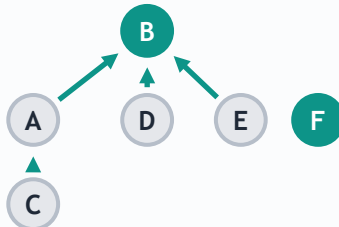
C aponta para A

UNION(B,C) · B-C (4)



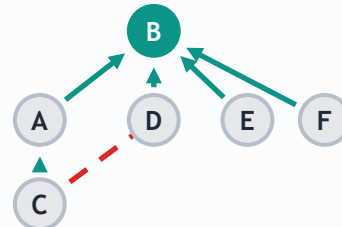
duas ilhas viram uma

UNION(D,E) · D-E (4)



E entra na ilha de B

C-D (5): FIND iguais → ciclo



D-F (5) une a última ilha

FIND com compressão de caminho

O **FIND** sobe do nó até a raiz. A cada passo religa o nó ao **avô** (linha $\text{pai}[x] = \text{pai}[\text{pai}[x]]$), achatando a árvore — assim as próximas buscas ficam muito mais rápidas.

Antes — corrente longa

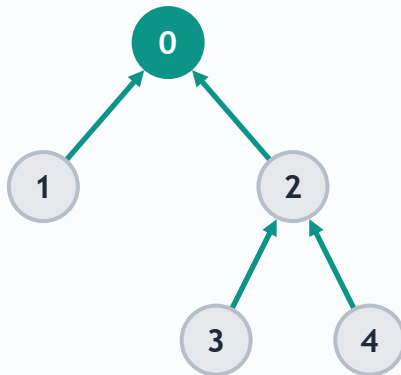


find(4): sobe 4 níveis

find(4)



Depois — árvore achatada



4 chega à raiz em 2 saltos

Passo a passo

```
pai = [0,0,1,2,3]
(corrente 4→3→2→1→0)
```

```
x=4: pai[4]=pai[2]=1; x=1
```

```
x=1: pai[1]=pai[0]=0; x=0
```

```
x=0: pai[0]=0 → raiz!
```

```
retorna 0
```

Cada nó visitado passa a apontar para o avô. O caminho encolhe e o representante (raiz **0**) continua o mesmo.

ALGORITMO 2

Algoritmo de Prim

Estratégia gulosa orientada a vértices, com fila de prioridade (heap).

Definição e regras

Ideia central: faça a árvore **crescer a partir de um vértice**, adicionando sempre a aresta mais barata que liga a árvore a um vértice ainda de fora.

- 1 Escolher a raiz** — Pegue um vértice inicial qualquer e marque-o como visitado.
- 2 Olhar a fronteira** — Considere todas as arestas que saem da árvore para vértices de fora.
- 3 Selecionar a menor** — Escolha a aresta de menor peso dessa fronteira (corte).
- 4 Incorporar vértice** — Adicione a aresta e o novo vértice à árvore; atualize a fronteira.
- 5 Repetir** — Continue até que todos os $|V|$ vértices estejam na árvore.

Estrutura-chave: Fila de prioridade

Um min-heap entrega sempre o vértice de fora mais barato de alcançar.

chave[v] = menor peso de aresta que liga v à árvore atual.

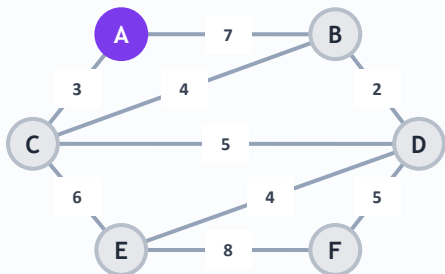
Sempre conexo

Diferente do Kruskal, a estrutura intermediária é sempre uma única árvore conectada que vai crescendo.

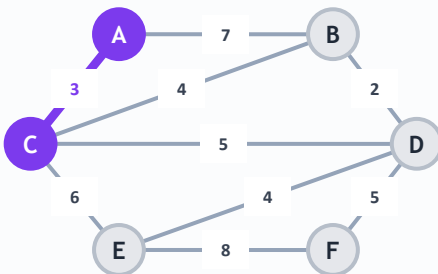
Execução em um grafo

Partindo do vértice A · a cada passo entra a aresta mais barata da fronteira

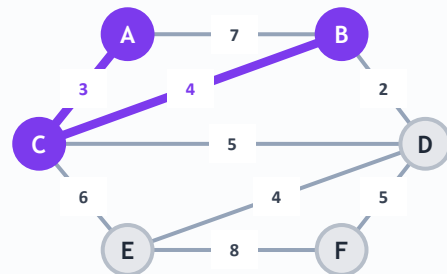
Passo 0 — raiz A



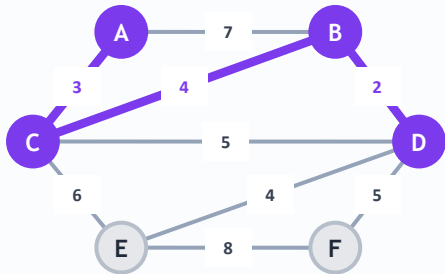
① A-C (3)



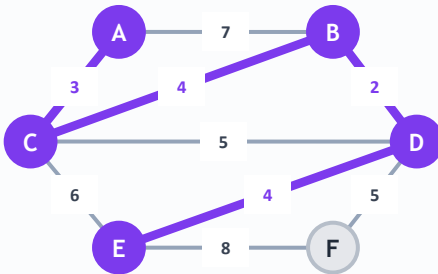
② C-B (4)



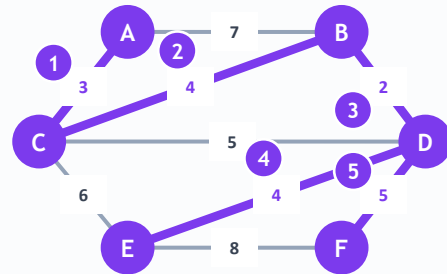
③ B-D (2)



④ D-E (4)



⑤ D-F (5) · peso = 18



Pseudocódigo e complexidade

Pseudocódigo

```
PRIM( $G = (V, A)$ , raiz  $r$ ):  
  para cada  $v$  em  $V$ :  
    chave[ $v$ ]  $\leftarrow \infty$ ; pai[ $v$ ]  $\leftarrow$  nulo  
  chave[ $r$ ]  $\leftarrow 0$   
   $Q \leftarrow$  fila de prioridade com todos os  $v$  (por chave)  
  enquanto  $Q$  não vazia:  
     $u \leftarrow \text{EXTRACT-MIN}(Q)$  // menor da fronteira  
    para cada vizinho  $v$  de  $u$ :  
      se  $v \in Q$  e peso( $u, v$ )  $<$  chave[ $v$ ]:  
        pai[ $v$ ]  $\leftarrow u$   
        chave[ $v$ ]  $\leftarrow$  peso( $u, v$ ) // DECREASE-KEY  
  retornar { ( $v$ , pai[ $v$ ]) }
```

Complexidade

Heap binário	$O(A \log V)$
--------------	---------------

Heap de Fibonacci	$O(A + V \log V)$
-------------------	-------------------

Matriz / array simples	$O(V^2)$
------------------------	----------

Qual usar? Com heap binário, $O(A \log V)$ é o padrão. Em grafos muito densos ($A \approx V^2$) a versão com matriz $O(V^2)$ pode ser mais simples e competitiva.

Kruskal × Prim

Característica	Kruskal	Prim
Abordagem	Orientado a arestas (global)	Orientado a vértices (cresce a partir de um nó)
Estrutura de apoio	Union-Find (DSU)	Fila de prioridade (heap)
Estado intermediário	Floresta de árvores que se fundem	Uma única árvore conexa
Complexidade	$O(A \log A) \approx O(A \log V)$	$O(A \log V)$ com heap binário
Melhor cenário	Grafos esparsos	Grafos densos (versão $O(V^2)$)

Em comum: ambos são gulosos, produzem uma AGM de mesmo peso total e, se os pesos forem todos distintos, a AGM é única (encontram exatamente a mesma árvore).

Kruskal

- + Simples com arestas já ordenadas.
- + Ótimo para grafos esparsos.
- Ordenar todas as arestas custa caro.
- Exige Union-Find eficiente.

Prim

- + Eficiente em grafos densos.
- + Não precisa ordenar arestas.
- Precisa de heap / decrease-key.
- Mais complexo de implementar bem.

Kruskal em Python

```
def kruskal(n, arestas):
    # arestas: lista de (peso, u, v)
    pai = list(range(n))

    def find(x):
        while pai[x] != x:
            pai[x] = pai[pai[x]]    # path compression
            x = pai[x]
        return x

    def union(a, b):
        pai[find(a)] = find(b)

    agm, custo = [], 0
    for peso, u, v in sorted(arestas):
        if find(u) != find(v):    # nao forma ciclo
            union(u, v)
            agm.append((u, v, peso))
            custo += peso
    return agm, custo
```

Como funciona

sorted(arestas) ordena por peso (1º item da tupla). find/union mantêm as componentes; só entram arestas que ligam componentes diferentes.

Exemplo de uso

```
arestas = [
    (7,0,1),(3,0,2),(4,1,2),
    (2,1,3),(5,2,3),(6,2,4),
    (4,3,4),(5,3,5),(8,4,5)]
kruskal(6, arestas)
# custo = 18
```


Prim em Python

```
import heapq

def construir(arestas, n):      # lista de adjacencia
    adj = [[] for _ in range(n)]
    for peso, u, v in arestas:
        adj[u].append((peso, v)) # aresta u-v...
        adj[v].append((peso, u)) # ...nos dois sentidos
    return adj

def prim(n, adj, raiz=0):
    visitado = [False] * n
    agm, custo = [], 0
    heap = [(0, -1, raiz)]      # (peso, origem, destino)
    while heap:
        peso, u, v = heapq.heappop(heap)
        if visitado[v]:
            continue            # ja esta na arvore
        visitado[v] = True
        if u != -1:
            agm.append((u, v, peso)); custo += peso
        for w, viz in adj[v]:
            if not visitado[viz]:
                heapq.heappush(heap, (w, v, viz))
    return agm, custo
```

Como funciona

construir() lista os vizinhos de cada vértice — cada aresta entra nos dois sentidos. No prim, o min-heap devolve sempre a aresta mais barata da fronteira; visitado[v] evita ciclos.

Exemplo de uso

```
adj = construir(arestas, 6)
prim(6, adj, raiz=0)
# custo = 18
# mesma AGM do Kruskal
```

PRÁTICA

Exercícios de simulação

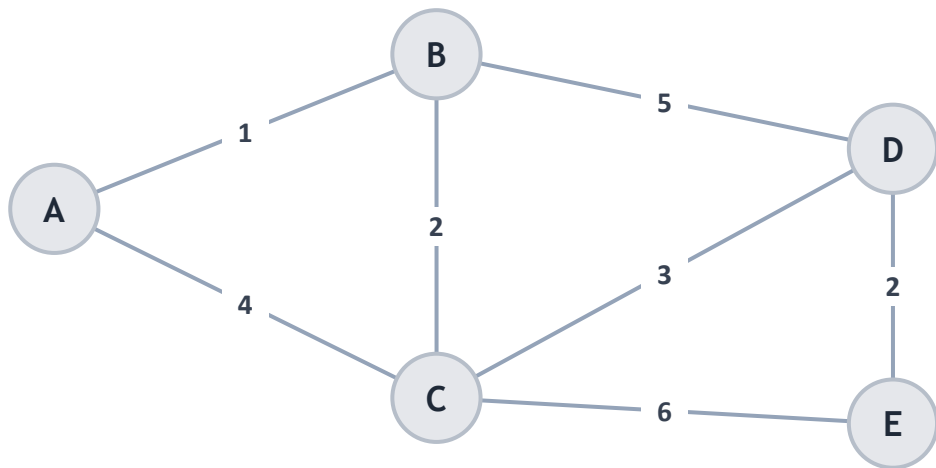
Aplique Kruskal ou Prim e acompanhe a construção da AGM passo a passo.

EXERCÍCIO 1 · SIMULAÇÃO

Algoritmo de Kruskal

Encontre a AGM do grafo de 5 vértices abaixo (sem ciclos rejeitados antes de completar).

Grafo

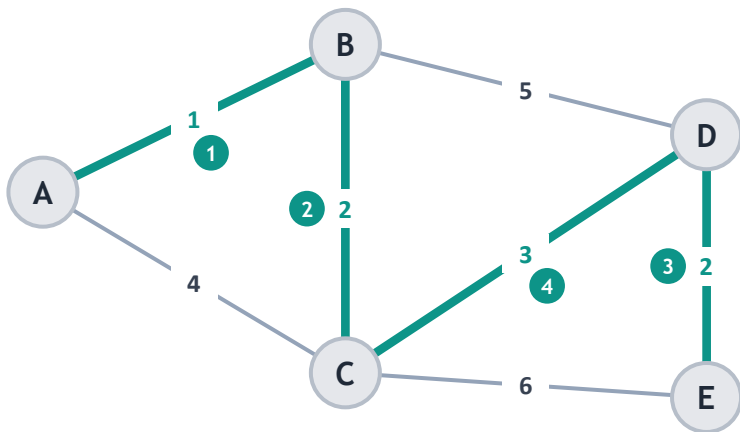


Pede-se

Ordene as arestas por peso e use o teste de ciclo (Union-Find). Anote as arestas que entram na AGM, na ordem, e o peso total.

Algoritmo de Kruskal

Esquema visual — números = ordem de seleção



Passo a passo

#	Aresta	Peso	Ação
1	A-B	1	adiciona
2	B-C	2	adiciona
3	D-E	2	adiciona
4	C-D	3	adiciona ✓ completa

Resposta

Ordem: A-B, B-C, D-E, C-D

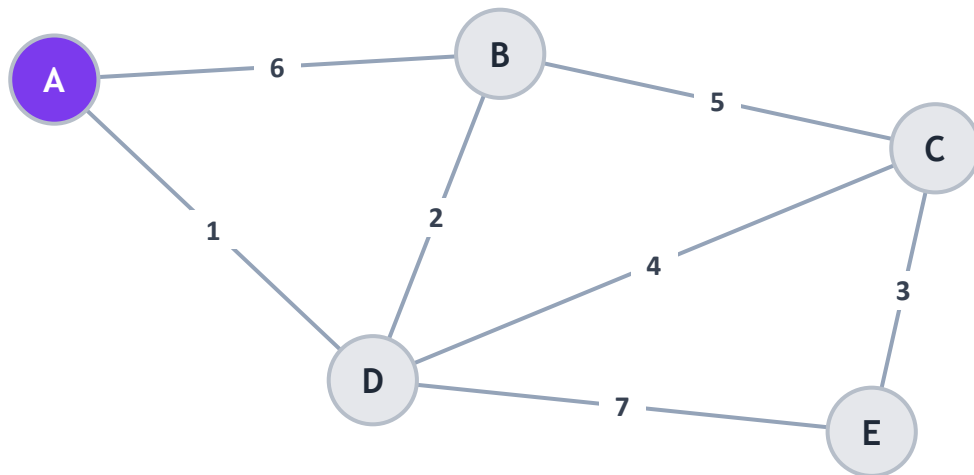
Peso total da AGM = 8

EXERCÍCIO 2 · SIMULAÇÃO

Algoritmo de Prim (raiz = A)

Construa a AGM pelo algoritmo de Prim, começando pelo vértice A.

Grafo

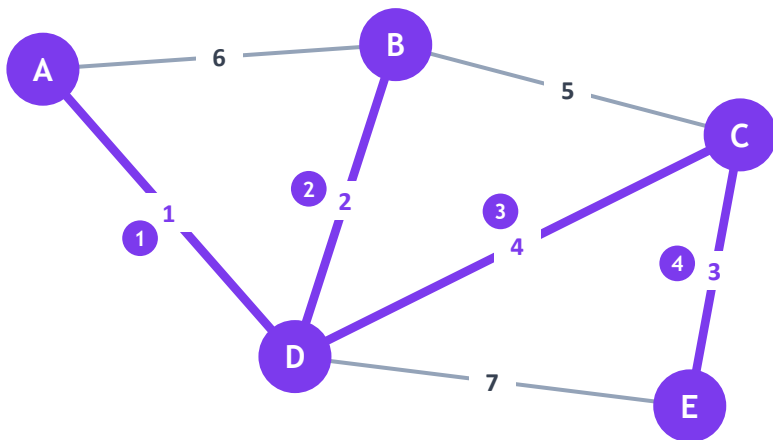


Pede-se

Partindo da raiz A, a cada passo escolha a aresta mais leve que liga a árvore a um vértice ainda de fora. Anote a ordem das arestas e o peso total.

Algoritmo de Prim (raiz = A)

Esquema visual — números = ordem de seleção



Passo a passo

#	Aresta	Peso	Ação
1	A-D	1	entra
2	D-B	2	entra
3	D-C	4	entra
4	C-E	3	entra ✓ completa

Resposta

Ordem: A-D, D-B, D-C, C-E

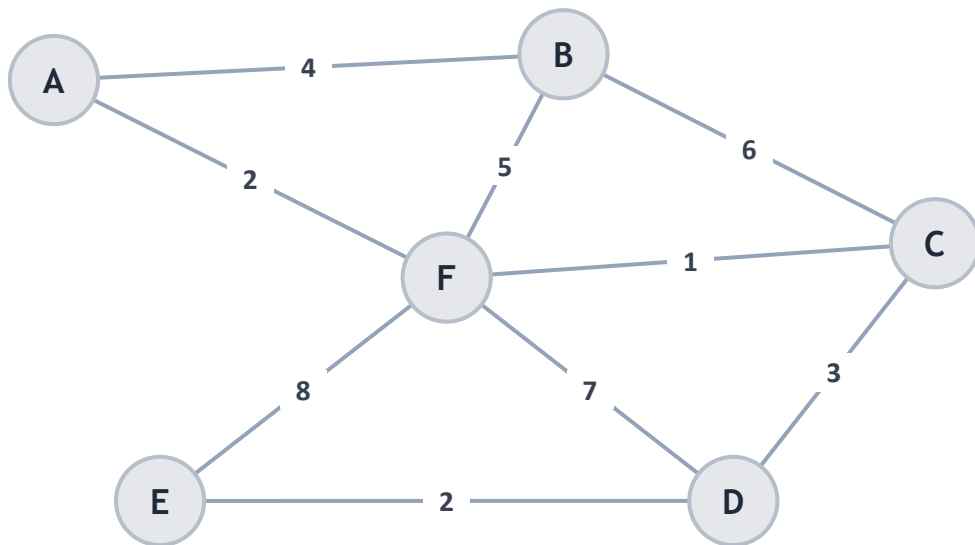
Peso total da AGM = 10

EXERCÍCIO 3 · SIMULAÇÃO

Algoritmo de Kruskal

Aplique Kruskal ao grafo de 6 vértices (F é um vértice central, bastante conectado).

Grafo

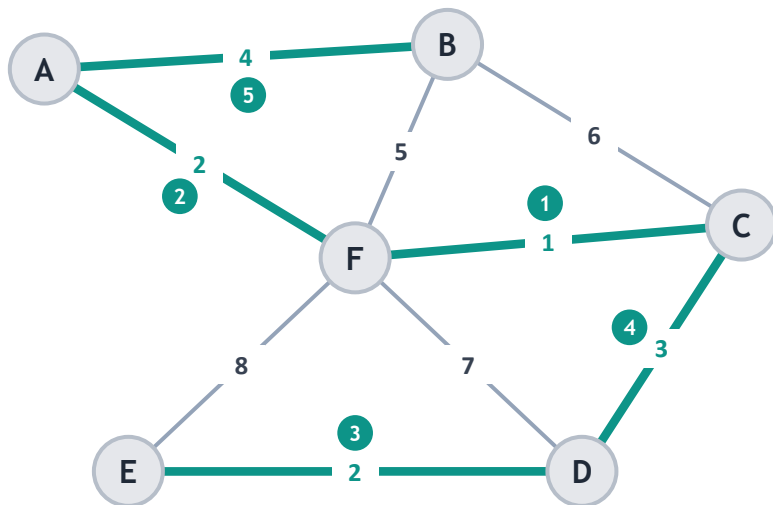


Pede-se

Ordene as arestas por peso e use o teste de ciclo (Union-Find). Anote as arestas que entram na AGM, na ordem, e o peso total.

Algoritmo de Kruskal

Esquema visual — números = ordem de seleção



Passo a passo

#	Aresta	Peso	Ação
1	C-F	1	adiciona
2	A-F	2	adiciona
3	D-E	2	adiciona
4	C-D	3	adiciona
5	A-B	4	adiciona ✓ completa

Resposta

Ordem: C-F, A-F, D-E, C-D, A-B

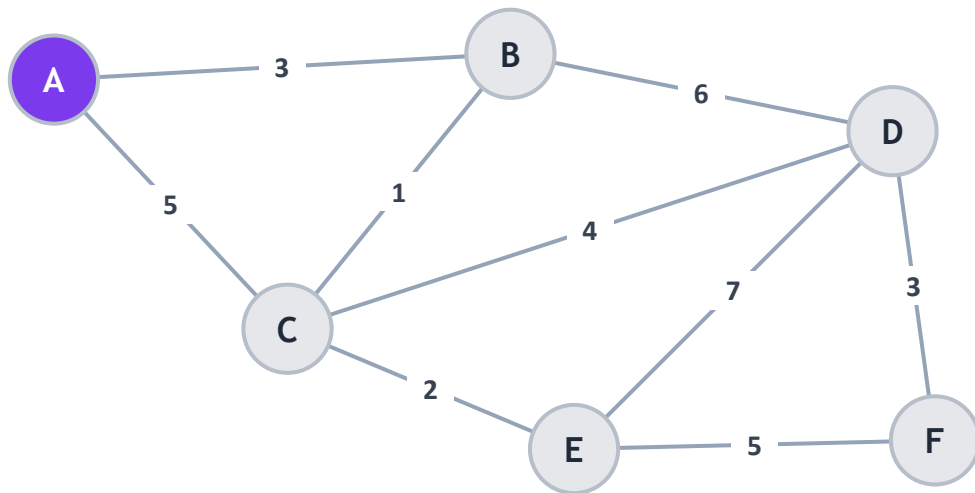
Peso total da AGM = 12

EXERCÍCIO 4 · SIMULAÇÃO

Algoritmo de Prim (raiz = A)

Construa a AGM por Prim a partir de A. Observe como a fronteira muda a cada passo.

Grafo



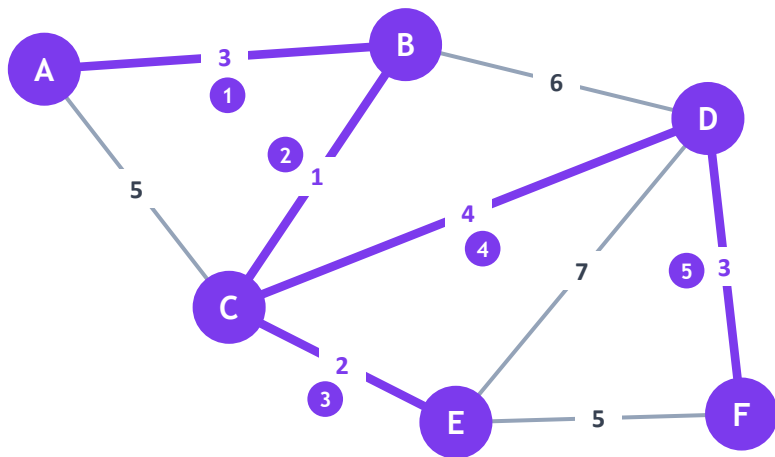
Pede-se

Partindo da raiz A, a cada passo escolha a aresta mais leve que liga a árvore a um vértice ainda de fora. Anote a ordem das arestas e o peso total.

EXERCÍCIO 4 · GABARITO

Algoritmo de Prim (raiz = A)

Esquema visual — números = ordem de seleção



Passo a passo

#	Aresta	Peso	Ação
1	A-B	3	entra
2	B-C	1	entra
3	C-E	2	entra
4	C-D	4	entra
5	D-F	3	entra ✓ completa

Resposta

Ordem: A-B, B-C, C-E, C-D, D-F

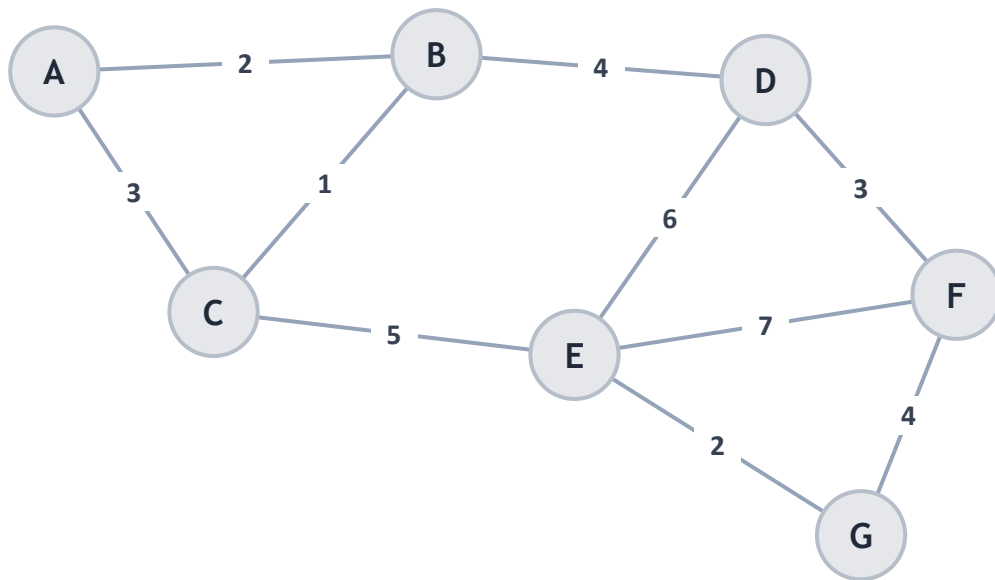
Peso total da AGM = 13

EXERCÍCIO 5 · SIMULAÇÃO

Algoritmo de Kruskal

Grafo de 7 vértices. Atenção: uma das arestas fecha ciclo e deve ser rejeitada.

Grafo

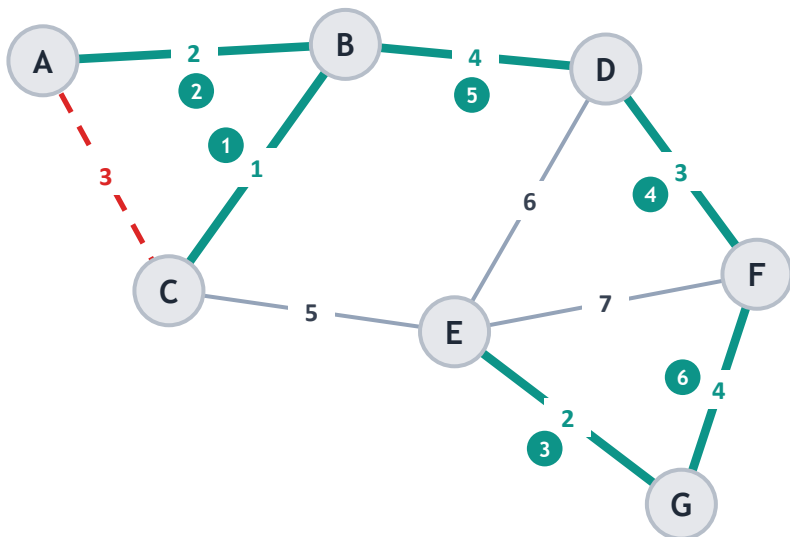


Pede-se

Ordene as arestas por peso e use o teste de ciclo (Union-Find). Anote as arestas que entram na AGM, na ordem, e o peso total.

Algoritmo de Kruskal

Esquema visual — números = ordem de seleção



Passo a passo

#	Aresta	Peso	Ação
1	B-C	1	adiciona
2	A-B	2	adiciona
3	E-G	2	adiciona
—	A-C	3	rejeita (ciclo)
4	D-F	3	adiciona
5	B-D	4	adiciona
6	F-G	4	adiciona ✓ completa

Resposta

Ordem: B-C, A-B, E-G, D-F, B-D, F-G

Peso total da AGM = 16

APLICAÇÃO

Exercícios contextualizados

Problemas reais que se reduzem a encontrar uma Árvore Geradora Mínima.

EXERCÍCIO 1 · CONTEXTUALIZADO

Rede de fibra óptica municipal

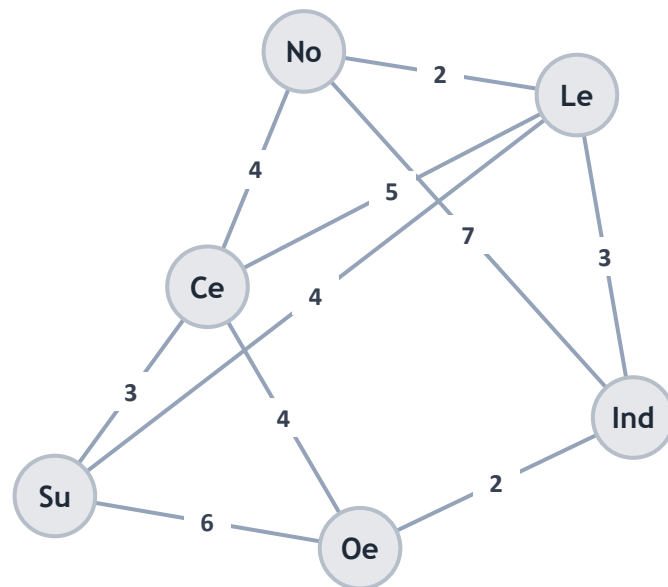
Cenário

A prefeitura vai levar fibra óptica a 6 regiões: Centro (Ce), Norte (No), Sul (Su), Leste (Le), Oeste (Oe) e o Distrito Industrial (Ind). Os pesos das arestas são a distância de cabo (em km) entre regiões viáveis de interligar.

Pergunta

Qual o traçado que conecta TODAS as regiões usando o MENOR comprimento total de cabo? Quantos km serão necessários?

Grafo do problema



EXERCÍCIO 1 · GABARITO

Rede de fibra óptica municipal

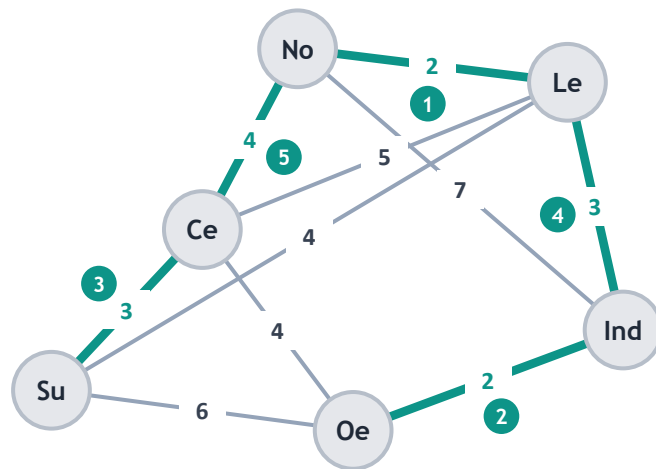
Cenário

A prefeitura vai levar fibra óptica a 6 regiões: Centro (Ce), Norte (No), Sul (Su), Leste (Le), Oeste (Oe) e o Distrito Industrial (Ind). Os pesos das arestas são a distância de cabo (em km) entre regiões viáveis de interligar.

Pergunta

Qual o traçado que conecta TODAS as regiões usando o MENOR comprimento total de cabo? Quantos km serão necessários?

Solução por Kruskal – AGM destacada



Resposta: No-Le, Oe-Ind, Ce-Su, Le-Ind, Ce-No. **Total = 14 km**

EXERCÍCIO 2 · CONTEXTUALIZADO

Rede de abastecimento de água

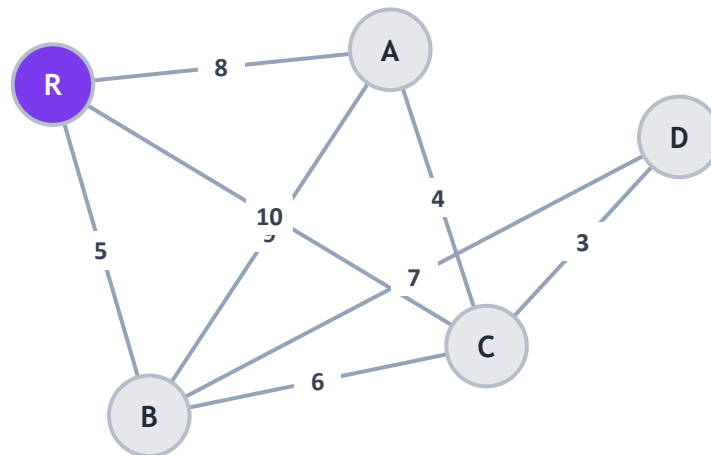
Cenário

Um Reservatório (R) deve abastecer 4 estações de bombeamento (A, B, C, D). Os pesos são o custo (em milhares de R\$) de instalar a tubulação em cada trecho possível. Toda estação precisa receber água, direta ou indiretamente, a partir de R.

Pergunta

Quais trechos de tubulação devem ser construídos para conectar todas as estações ao reservatório com o MENOR custo total?

Grafo do problema (raiz R)



EXERCÍCIO 2 · GABARITO

Rede de abastecimento de água

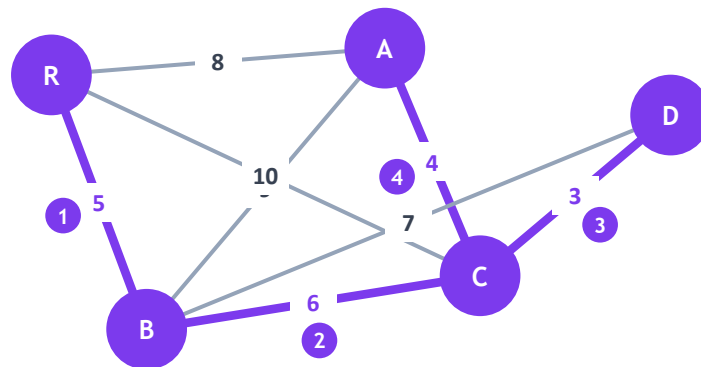
Cenário

Um Reservatório (R) deve abastecer 4 estações de bombeamento (A, B, C, D). Os pesos são o custo (em milhares de R\$) de instalar a tubulação em cada trecho possível. Toda estação precisa receber água, direta ou indiretamente, a partir de R.

Pergunta

Quais trechos de tubulação devem ser construídos para conectar todas as estações ao reservatório com o MENOR custo total?

Solução por Prim (raiz R) — AGM destacada



Resposta: R-B, B-C, C-D, C-A. **Total = R\$ 18 mil**

EXERCÍCIO 3 · CONTEXTUALIZADO

Cabeamento de parque eólico

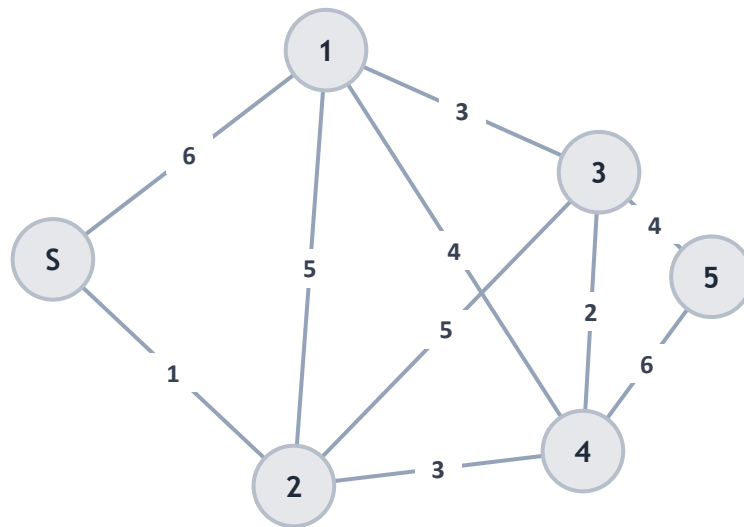
Cenário

Um parque eólico tem uma Subestação (S) e 5 turbinas (1 a 5). É preciso interligar todas as turbinas à subestação por cabos subterrâneos. Os pesos representam o custo de cabeamento de cada trecho viável. Uma das ligações forma ciclo e deve ser descartada.

Pergunta

Defina o conjunto de cabos de menor custo que conecta todas as turbinas à subestação. Qual aresta é rejeitada por formar ciclo?

Grafo do problema



Cabeamento de parque eólico

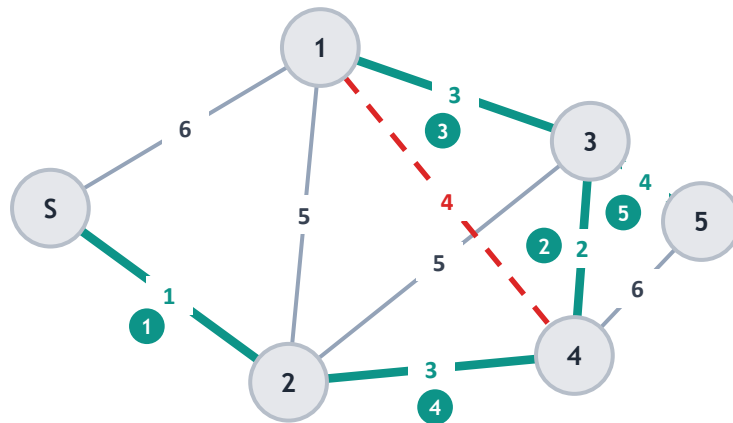
Cenário

Um parque eólico tem uma Subestação (S) e 5 turbinas (1 a 5). É preciso interligar todas as turbinas à subestação por cabos subterrâneos. Os pesos representam o custo de cabeamento de cada trecho viável. Uma das ligações forma ciclo e deve ser descartada.

Pergunta

Defina o conjunto de cabos de menor custo que conecta todas as turbinas à subestação. Qual aresta é rejeitada por formar ciclo?

Solução por Kruskal — AGM destacada



Resposta: S-2, 3-4, 1-3, 2-4, 3-5 (rejeita 1-4). **Total = 13**

SÍNTESE

O essencial sobre AGM

AGM

Subgrafo acíclico que conecta todos os vértices com peso total mínimo ($|V| - 1$ arestas).

Kruskal

Guloso por arestas + Union-Find. $O(A \log A)$. Brilha em grafos esparsos.

Prim

Guloso por vértices + heap. $O(A \log V)$. Brilha em grafos densos.

Resultado

Ambos produzem AGM de mesmo peso; única quando os pesos são distintos.